

Isaac Hermel Reginato

isaachermel@gmail.com | +55 51 99709-9876 | Porto Alegre, RS, Brazil (Remote/Hybrid) | linkedin.com/in/isaachermel | github.com/h3rmel | isaachermel.dev.br

PROFESSIONAL SUMMARY

Full-stack Software Engineer with 4 years of experience in the JavaScript/TypeScript ecosystem. Maintainer of an open-source library with ~102 stars distributed via the shadcn registry (guarahooks — 50 React hooks) and contributor to a production design system (StencilJS/Storybook). End-to-end work: front-end in React/Next.js with a focus on performance (Core Web Vitals) and back-end in NestJS/Node.js, Python, and .NET with REST APIs, orchestrated ETL pipelines (Apache Airflow), and rate-limited integrations, in layered architecture with dependency injection. Author of an internal library consumed by four applications, and facilitator of internal technical DOJOs for standards dissemination. Shift-Left testing strategies (TDD/BDD) and delivery under feature flags.

CORE COMPETENCIES

System Design | Layered Architecture | Dependency Injection | Internal Library Design | Performance / Core Web Vitals | SSR/SSG | Shift-Left Testing (TDD/BDD) | REST APIs | Messaging & Data Processing | Design Systems | CI/CD

TECHNICAL SKILLS

- **Languages:** JavaScript, TypeScript, Python, C#;
- **Front-end:** React.js, Next.js, Vue, Blazor, Tailwind CSS, Framer Motion, Bootstrap, offline-first PWA;
- **State & Data (front):** Zustand, Context API, Axios, RESTful API consumption, real-time communication (WebSockets/SSE);
- **Design Systems:** Storybook, StencilJS (Web Components), shadcn package registry, content-collections;
- **Back-end & Integrations:** Node.js, Express.js, NestJS, Flask, .NET; authentication flows (OAuth 2.0, JWT); data ingestion and processing pipelines (ETL), Apache Airflow, ETL orchestration;
- **Databases & Messaging:** PostgreSQL, MongoDB (schema modeling and migrations), Redis, Kafka;
- **Cloud, DevOps & Observability:** AWS (S3, CloudFront), GCP (Firebase, Cloud Run), Kubernetes (deployment manifests for owned services), Docker, Argo CD, GitHub Actions, GitLab CI/CD, Grafana (alert integration);
- **Testing:** Vitest, Jest, Testing Library; TDD and BDD methodologies;
- **Architecture & Practices:** SOLID, layered architecture, dependency injection, Result Pattern, TDD, BDD, MVC, MVVM;
- **Tools:** Git (GitFlow), Figma; AI-assisted development with a spec-driven workflow (OpenSpec), rules and specs versioned in-repository — applied across three public repositories (guarahooks, enterprise-react-architecture, distributed-systems-lab).

PROFESSIONAL EXPERIENCE

Software Engineer, Zenvia — 08/2023 – Present

- **Ownership (Back-end):** Ownership of `partners-api` — a batch data-import service via reading, extraction, treatment, and processing of CSV files, structured in layered architecture with dependency injection to isolate business rules from I/O boundaries; sustaining peaks of 60k rows/minute in production;
- **Ownership (Integrations & Data):** Ownership of the ETL pipelines synchronizing with ERPs (Bling, Tiny, Microvix) — Apache Airflow DAGs (Python) covering contacts, orders, invoices, and product catalog, with a full initial load and incremental syncs. Architecture designed against two nested vendor-API budgets (3 req/s and 120k req/day, per client) under an HTTP N+1 pattern: work-conserving scheduling with priority-weighted allocation across concurrent streams and durable resumption via checkpoint on hitting the daily quota. Took ownership following the optimization contribution;

- **Ownership (Internal libraries):** Authorship of `security-settings-sdk` — an authenticated HTTP client with a cache layer for the anti-phishing settings service, consumed by four applications (URL Manager, Templates Management, api-control, and api-subscription-control), removing duplicated client and cache logic across those repositories. Intentionally thin: owns no infrastructure, receives the cache as an injected port and its timeout budgets from the caller, and applies no fail-open/fail-closed policy of its own — it reports the condition and keeps the decision visible in consumer code. Errors surfaced as a typed discriminated union (Result Pattern), separating upstream, authorization, and contract failures; dual-TTL cache with synchronous revalidation, per-key single-flight deduplication of concurrent lookups, and delete-based invalidation instead of write-through, given no ordering guarantee across four consumers of the same Kafka topic;
- **Logger library (Node.js):** Fixed an instance-identity regression in the organization's logging library — exporting via object spread severed the `winston.Logger` → `stream.Transform` → `events.EventEmitter` prototype chain, breaking third-party integrations, `instanceof` checks, test context isolation, and mocking. Replaced with structured patching over the native instance, intercepting `.log()` for validation, immutability, and context propagation via `AsyncLocalStorage`, while preserving a bound reference to the original method to prevent recursion under spying. Extended unit and integration suites to full coverage;
- **Data & Operations (WhatsApp messaging):** Coordinated the schema update and migration of the WhatsApp MongoDB collections to incorporate Meta's new BSUID (Business-Scoped User ID) identifiers, issued to identify users on the WhatsApp Business API. Connected Grafana to Google Chat as the team's operational alerting channel, routing alerts on platform and business metrics — Kafka consumer lag, WhatsApp account open/close events, and phishing suspicion — into actionable notifications;
- **Technical dissemination & governance:** Ran internal DOJOs — Result Pattern, subsequently adopted as the error-handling standard in `security-settings-sdk`, and an instrumented comparison of Winston vs. PinoJS with measured results, informing the decision on a new logging library. Standardized documentation generation for internal libraries with JSDoc and TypeDoc, producing a consistent README per library, structured for consumption by developers and AI tooling; documentation templates and a changelog standard adopted across multiple projects; Prettier and ESLint enforced in the GitLab CI pipeline;
- **Front-end (Design System & Performance):** Contribution to the company design system in StencilJS (Web Components), using Storybook for isolated component development and documentation. Core Web Vitals optimization in production (field metrics): LCP reduced from 2.91s to 2.35s and INP from 275ms to 199ms, both moved from the "needs improvement" range into "good", via rendering refactor in React.js with the MVVM pattern.

Front-end Developer, Catency (Freelance / Contractor) — 04/2023 – 03/2025

- **Context:** Engineering of high-conversion landing pages and institutional websites;
- **Architecture:** Development with Next.js and Tailwind CSS applying hybrid rendering (SSG/SSR) for indexing control (SEO) and static-asset delivery optimization. Continuous Lighthouse audits for performance and accessibility (a11y/ARIA);
- **Impact:** Delivery of 5 platforms with consistent LCP < 2.5s and Core Web Vitals thresholds met for strong organic indexing.

Full-stack Developer, Umbler — 02/2022 – 04/2023

- **Context:** Creation of internal tools for Customer Success operations and portfolio management;
- **Architecture:** End-to-end development with .NET and Blazor on the MVC pattern, separating data modeling from control interfaces. Customization of the Bootstrap library as the standardized Design System across internal systems;
- **Impact:** Enabled process documentation, learning tracks, and automated management of a 100,000-client base.

PROJECTS & OPEN-SOURCE CONTRIBUTIONS

Distributed Systems Lab

- High-performance full-stack monorepo: webhook ingestion API (NestJS/Fastify, BullMQ), stream data-processing pipeline, and real-time analytics dashboard (Next.js, Socket.io, Zustand);
- Processing of 5 million records with 57MB of RAM (Node.js Streams + Postgres COPY); front-end at 60 FPS under high-frequency WebSocket events; API with 0% error rate and 7.35ms latency (P95) under 500 concurrent users.

Guarahooks

- Open-source library with 50 React hooks, ~102 GitHub stars, distributed via the shadcn registry and with a CLI published on npm (`guara-cli`). Documentation ecosystem integrated with content-collections;
- Maintenance of code consumed by third-party applications.

Enterprise React Architecture

- Reference implementation for mission-critical React applications. "Shift-Left Testing" approach with TypeScript, Vitest, and React Testing Library, applying TDD/BDD to validate complex UI domains;
- Scalable model enabling zero-regression deployment pipelines.

MagicUI

- Development of 5 visual components with React.js, Tailwind CSS, and Framer Motion. Lazy Loading and memoization (useMemo, useCallback) for bundle and component-lifecycle optimization.

EDUCATION

- **Postgraduate, Software Engineering** — in progress (started 2026);
- **Systems Analysis and Development** — Centro Universitário União das Américas Descomplica (2023 – 2025, completed);
- **Internet Computing (Technical)** — Instituto Federal Sul-rio-grandense (2018 – 2022);
- **Continuing education:** curso.dev (in progress).

LANGUAGES

- English: Advanced / Fluent technical reading
- Portuguese: Native